Advanced Programming

Spring Framework (REST Services)

The Context

- Most applications use various technologies (GUI, ORM, services, logging, testing, DI, etc)
- Integrating all these technologies may be a difficult and repetitive task
- The common usage of some technologies may be simplified (*implicit middleware*)
- We want to expose **network services**.
- We need:
  - application frameworks or/and
  - application servers

Application Server

- An *application server* hosts applications
- An *application* contains *components*
  - UI components (for creating presentation)
  - Bussines components (for implementing logic)
  - Services (for communication)
  - Administrative objects: data sources, queues, etc.
- Components exist inside *containers*
  - Web, EJB, IOC, etc.
- An AS implements *specifications (Java EE)*
- Tomcat, Glassfish, WildFly, TomEE ,etc.

Application Framework

- Simplifies the creation of an *application*
- Provides *modules* for:
  - Logging, Testing, Data access, Security
  - MVC, AOP, IOC, etc.
- A *lightweight* approach to the functionalities offered by the *heavier* application servers
- For Web functionalities, they still need an *embedded server* (small, light).
- SpringFramework, Apache Tapestry, Google Guice, etc.

Network Services

- Network services use *protocols*
- Transport protocols:
  - TCP, UDP, etc. → Data transfer
- Application protocols (on top of transport):
  - **HTTP**, HTTPS → Data representation
- How to expose/communicate with a service?
  - Rigurous (heavy) model
    - Describe: **WSDL**, etc
    - Locate: **UDDI**, etc.
    - Invoke (communicate): **SOAP**, etc.
  - Simple model: **REST** paradigm

Spring Framework

spring.io

- “Spring makes programming Java quicker, easier, and safer.”
- Focus on speed, simplicity, and productivity
- Developed by Pivotal Software (2002→ today)
- The world's most popular Java framework
  - Intersects with and offers alternatives to Java EE
- Organized in a modular fashion
  - Web apps, Data access, Micro services, etc.

Spring Boot

- Spring Boot makes it easy to create **stand-alone** applications that you can "just run".
- Simple way to create (Micro)Services.
- Embed Tomcat, Jetty or Undertow directly (no need to deploy WAR files)
- **Spring initializr**: https://start.spring.io/
  - Choose the language: Java, Kotlin, Groovy,
  - Build system: Maven, Gradle,
  - JVM version, Spring version, Dependencies

Let's look at *pom.xml*

```
...
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    ...
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

Starter for building web, including RESTful, applications using Spring MVC. Uses Tomcat as the default embedded container

The *main* class

```
package com.example.demo;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
        System.out.println("Hello world!");
    }
}
```

The entry point of the Spring Boot Application is the class containing the @SpringBootApplication annotation. This class should have the *main* method to run the Spring Boot application.
@SpringBootApplication annotation includes Auto-Configuration, Component Scan, and Spring Boot Configuration.

“Running” the application

```
:: Spring Boot ::        (v2.2.6.RELEASE)
: Starting DemoApplication on Cristi-PC with PID 5232
: Tomcat initialized with port(s): 8080 (http)
: Starting Servlet engine: [Apache Tomcat/9.0.33]
: Initializing Spring embedded WebApplicationContext
: Initializing ExecutorService 'applicationTaskExecutor'
: Tomcat started on port(s): 8080 (http) with context path ''
: Started DemoApplication in 1.994 seconds
Hello world!
```

Creating a simple REST service

```
package com.example.demo;

@RestController
public class HelloController {

    @RequestMapping("/hello")
    public String sayHello() {
        return "Greetings from Spring Boot!";
    }
}
```

http://localhost:8080/hello
Greetings from Spring Boot!

The Spring Web model-view-controller (MVC) framework is designed around a DispatcherServlet that dispatches requests to handlers, with configurable handler mappings.
The default handler is based on the @Controller and @RequestMapping annotations.

Creating a simple test

```
package com.example.demo;

@SpringBootTest
@AutoConfigureMockMvc
public class HelloControllerTest {

    @Autowired
    private MockMvc mvc;

    @Test
    public void getHello() throws Exception {
      mvc.perform(MockMvcRequestBuilders.get("/hello")
          .accept(MediaType.APPLICATION_JSON))
          .andExpect(status().isOk())
          .andExpect(
            content().string(equalTo("Greetings from Spring Boot!"))
          );
    }
}
```

Autowired: Spring's dependency injection.
This is an alternative to Java EE *Inject* annotation.

RESTful Web Services

- Service implementation over HTTP that conforms to the REST *architectural style*.
- Data and functionality are considered **resources** and are accessed using an **uniform interface**:
  - **Identification** → **URI**, for example: `http://example.com/resources`
  - **Representations** → **Standard MIME types**: `Text, JSON, XML, YAML,` etc.
  - **Operations** → **Standard HTTP methods**: `HTTP POST, GET, PUT, DELETE`

HTTP Methods

- GET: Used to request data from a specified resource. Should not produce any side effect.
- POST: Used to **create** a new child resource, at a *server* defined URL. POST is **non-idempotent** which means multiple requests will have different effects
- PUT: Used to **create or replace if exists** a resource, at a URL known by the *client*. **Idempotent**, which means multiple requests will have the same effect.
- PATCH: Used to **update** part of the resource at the *client* defined URL.
- DELETE: Used to **remove** a specified resource (from the database, for example)

JSON

- **J**ava **S**cript **O**bject **N**otation
- Format for storing and exchanging textual data
- Examples:

```
{"name": "Java", "age": 25, "parent": "Sun Microsystems"}
```

```
{"languages":[
    { "name":"Java", "age":25},
    { "name":"Python", "age":31},
    { "name":"C++", "age":36 }]
}
```

- MIME: "application/json"
- Parsers: GSON, Jackson, etc.
- JSON vs. XML

Spring REST API support

- **@RestController** – @Controller and @ResponseBody
- **@RequestMapping**(value, method)

  Mapping web requests onto methods in request-handling classes with flexible method signatures.

  @GetMapping, @PostMapping, @PutMapping, @DeleteMapping
- **@RequestBody**: HttpRequest body → domain object
- **@ResponseBody**: domain object → HttpResponse
- **@PathVariable**: for custom or dynamic request URI
- **@RequestParam**: read a param from the request URI
- **ResponseEntity**: Builder for creating the response

GET

@RequestMapping(value = "/path", method = RequestMethod.GET, produces = "application/json")

```
@RestController
@RequestMapping("/products")
public class ProductController {

    private final List<Product> products = new ArrayList<>();
    public ProductController() {
        products.add(new Product(1, "Mask"));
        products.add(new Product(2, "Gloves"));
    }

    @GetMapping
    public List<Product> getProducts() {
        return products;
    }
    @GetMapping("/count")
    public int countProducts() {
        return products.size();
    }
    @GetMapping("/{id}")
    public Product getProduct(@PathVariable("id") int id) {
        return products.stream()
           .filter(p -> p.getId() == id).findFirst().orElse(null);
    }
}
```

localhost:8080/products

```
[{"id":1,"name":"Mask"},
{"id":2,"name":"Gloves"}]
```

localhost:8080/products/count

```
2
```

localhost:8080/products/1

```
{"id":1,"name":"Mask"}
```

POST

@RequestMapping(value = "/path", method = RequestMethod.POST)

@PostMapping is a composed annotation that acts as a shortcut for @RequestMapping(method = RequestMethod.POST)

```
@RestController
@RequestMapping("/products")
public class ProductController {
    ...
    @PostMapping
    public int createProduct(@RequestParam String name) {
      int id = 1 + products.size();
      products.add(new Product(id, name));
      return id;
    }

    @PostMapping(value = "/obj", consumes="application/json")
    public ResponseEntity<String>
                createProduct(@RequestBody Product product) {
        products.add(product);
        return new ResponseEntity<>(
          "Product created successfully", HttpStatus.CREATED);
    }
}
```

```
<html> <body>
  <form action="http://localhost:8080/products" method="POST">
    Product: <input type="text" name="name"/> <br/>
    <input type="submit" value="Submit"/>
  </form>
</bodY> </html>
```

PUT

@RequestMapping(value = "/path", method = RequestMethod.PUT)

```
@RestController
@RequestMapping("/products")
public class ProductController {
    ...
    @PutMapping("/{id}")
    public ResponseEntity<String> updateProduct(
                @PathVariable int id, @RequestParam String name) {
        Product product = findById(id);
        if (product == null) {
            return new ResponseEntity<>(
                "Product not found", HttpStatus.NOT_FOUND); //or GONE
        }
        product.setName(name);
        return new ResponseEntity<>(
                "Product updated successsfully", HttpStatus.OK);
    }
}
```

Postman:

PUT
http://localhost:8080/products/1
Query params:
key="name" value="The Mask of Zorro"

DELETE

@RequestMapping(value = "/path", method = RequestMethod.DELETE)

```
@RestController
@RequestMapping("/products")
public class ProductController {
    ...
    @DeleteMapping(value = "/{id}")
    public ResponseEntity<String> deleteProduct(@PathVariable int id) {
        Product product = findById(id);
        if (product == null) {
            return new ResponseEntity<>(
                "Product not found", HttpStatus.GONE);
        }
        products.remove(product);
        return new ResponseEntity<>("Product removed", HttpStatus.OK);
    }
}
```

Postman:

DELETE

http://localhost:8080/products/1

Naming Conventions

- The URL is a sentence, where resources are nouns and HTTP methods are verbs.
- The resource should always be plural.
- Specify an *id* to access one instance of the resource.
  - `GET /products`
  - `GET /products/123`
  - `POST /products`
  - `PUT /products/123`
  - `DELETE /products/123`
- Searching, sorting, filtering and pagination
  - `GET /products`**`?sort=name_asc`**
  - `GET /products?category=food&country=Romania`
  - `GET /products?search=Pizza`
  - `GET /products?page=2`
- Versioning `/products/v1`

HTTP Response Status Codes

- 2xx (Success category)
  - 200 **Ok**, 201 **Created**, 202 **Accepted**, 204 **No Content**
- 3xx (Redirection Category)
  - 301 **Moved Permanently**, 304 **Not Modified**
- 4xx (Client Error Category)
  - 400 **Bad Request**, 401 **Unauthorized**,
  - 403 **Forbidden**, 404 **Not Found**, 410 **Gone**
- 5xx (Server Error Category)
  - 500 **Internal Server Error**, 503 **Service Unavailable**

Calling a Service: RestTemplate

Synchronous client to perform HTTP requests.
Uses the Java Servlet API, which is based on the thread-per-request model.

```
@RestController
public class CallService {

    final Logger log = LoggerFactory.getLogger(CallService.class);
    final String uri = "http://localhost:8080/products";

    @GetMapping("/call")
    public List<Product> getProducts() {
        log.info("Start");
        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<List<Product>> response = restTemplate.exchange(
                uri, HttpMethod.GET, null,
                new ParameterizedTypeReference<List<Product>>(){});

        List<Product> result = response.getBody();
        result.forEach(p -> log.info(p.toString()));
        log.info("Stop");
        return result;
    }
}
```

RestTemplate.exchange executes the HTTP method to the given URI template, writing the given request entity to the request, and returns the response as ResponseEntity. The given ParameterizedTypeReference is used to pass generic type information

Calling a Service: WebClient

Asynchronous, non-blocking solution.
WebClient is part of the Spring WebFlux library.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

```
@GetMapping(value = "/flux",
        produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<Product> getProductsNonBlocking() {
    log.info("Start");
    Flux<Product> productFlux = WebClient.create()
            .get()
            .uri(uri)
            .retrieve()
            .bodyToFlux(Product.class);
    productFlux.subscribe(p -> log.info(p.toString()));

    log.info("Stop");
    return productFlux;
}
```

asynchronous data stream

Reactive programming is about non-blocking applications that are asynchronous and event-driven and require a small number of threads to scale vertically (i.e. within the JVM) rather than horizontally (i.e. through clustering)

Calling a Service: jQuery

jQuery is a lightweight, "write less, do more", JavaScript library.

```
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"/>
<script>
$(document).ready(function() {
    $.ajax({
        url: "http://localhost:8080/products/1"
    }).then(function(product) {
       $('.product-id').append(product.id);
       $('.product-name').append(product.name);
    });
});
</script>
</head>
<body>
  <div>
    <p class="product-id">The ID is </p>
    <p class="product-name">The name is </p>
  </div>
</bodY>
</html>
```

AJAX = **A**synchronous **J**avaScript **A**nd **X**ML

Creating a Web Page

- *Thymeleaf*: server-side Java template engine for both web and standalone environments. (default in Spring, can be replaced)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

- Put HTML pages in *resources/templates*
- Create a *Controller*

```
@Controller
public class PageController {
    @GetMapping(value = "/page")
    public String demoPage() {
        return "demo"; //demo.html must exist in resources/templates
    }
}
```

ModelAndView

- Model = data, View = page
- In HTML

```
<span th:text="'Hello, ' + ${user}"></span>
```

- In Controller

```
@GetMapping(value = "/page")
public ModelAndView demo() {
    ModelAndView modelAndView = new ModelAndView();
    modelAndView.setViewName("demo");
    modelAndView.addObject("user", "Duke");
    return modelAndView;
}
```

Exception Handling

- REST services may throw exceptions

```
@GetMapping(value = "/{id}")
public Product getProduct(@PathVariable("id") int id) {
  return products.stream()
   .filter(p -> p.getId() == id).findFirst()
   .orElseThrow( () -> new MyException("Product not found");
}
```

- Exception Handling via @ResponseStatus

```
@ResponseStatus(value = HttpStatus.NOT_FOUND, reason = "Karma")
public class MyException extends RuntimeException { … }
```

- Global Exception Handling

```
@RestControllerAdvice
public class MyExceptionAdvice {
  @ExceptionHandler(value = MyException.class)
  public ResponseEntity<MyErrorResponse>
       handleGenericNotFoundException(MyException e) {
    MyErrorResponse error = new MyErrorResponse(e.getMessage());
    error.setTimestamp(LocalDateTime.now());
    return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
  }
}
```

Interceptor Pattern

- A mechanism to change the usual communication between two components.
- Client → [Interceptor(s)] → Service
- Transparent, declarative
- Spring *InterceptorHandler*
  - *prehandle()* – called before the actual handler is executed, but the view is not generated yet
  - *postHandle()* – called after the handler is executed
  - *afterCompletion()* – called after the complete request has finished and view was generated

HTTPS

Hyper Text Transfer Protocol, Secure

- HTTPS = HTTP protocol over TLS/SSL
- We need a **SSL certificate** (digital certificate)
- SSL (Secure Sockets Layer) enables encrypted communication between a web browser and a web server.

  It authenticates the identity of the website and encrypts the data that's being transmitted
- Using Java **keytool** we can create a self-signed certificate:

```
keytool -genkeypair -alias demo -keyalg RSA -keysize 2048
        -storetype PKCS12 -keystore demo.p12 -validity 3650
```

- PKCS12: Public Key Cryptographic Standards

  JKS: Java KeyStore (limited to the Java environment)

Enabling HTTPS in Spring Boot

- Copy the SSL certificate *demo.p12* in *resources/keystore*
- Modify *application.properties* file

```
server.port: 443
server.ssl.key-store-type=PKCS12
server.ssl.key-store=classpath:keystore/demo.p12
server.ssl.key-store-password=spring
server.ssl.key-alias=demo
```

- Test your application

`https://localhost:443/products`

443 is used for secure web browser communication. Data transferred across such connections are highly resistant to eavesdropping and interception. Moreover, the identity of the remotely connected server can be verified with significant confidence. Web servers offering to accept and establish secure connections listen on this port for connections from web browsers desiring strong communication security.